7.2 Fully convolutional networks

- as previously discussed, semantic segmentation classifies images at pixel level
- a *fully convolutional network* (*FCN*) uses a convolutional neural network to transform image pixels to pixel classes
- unlike the CNNs that we encountered earlier for image classification or object detection, a fully convolutional network transforms the height and width of intermediate feature maps back to those of the input image: this is achieved by the transposed convolutional layer introduced in Section 7.1
- as a result, the classification output and the input image have a one-to-one correspondence at pixel level: the channel dimension at any output pixel holds the classification results for the input pixel at the same spatial position

7.2 Fully convolutional networks

- here, we describe the basic design of the fully convolutional network model
- as shown in Figure 3, this model first uses a CNN to extract image features, then transforms the number of channels into the number of classes via a 1×1 convolutional layer, and finally transforms the height and width of the feature maps to those of the input image via the transposed convolution introduced in Section 7.1
- as a result, the model output has the same height and width as the input image, where the output channel contains the predicted classes for the input pixel at the same spatial position
- upsampling is done by a transposed convolution initialized with bilinear interpolation

7.2 Fully convolutional networks

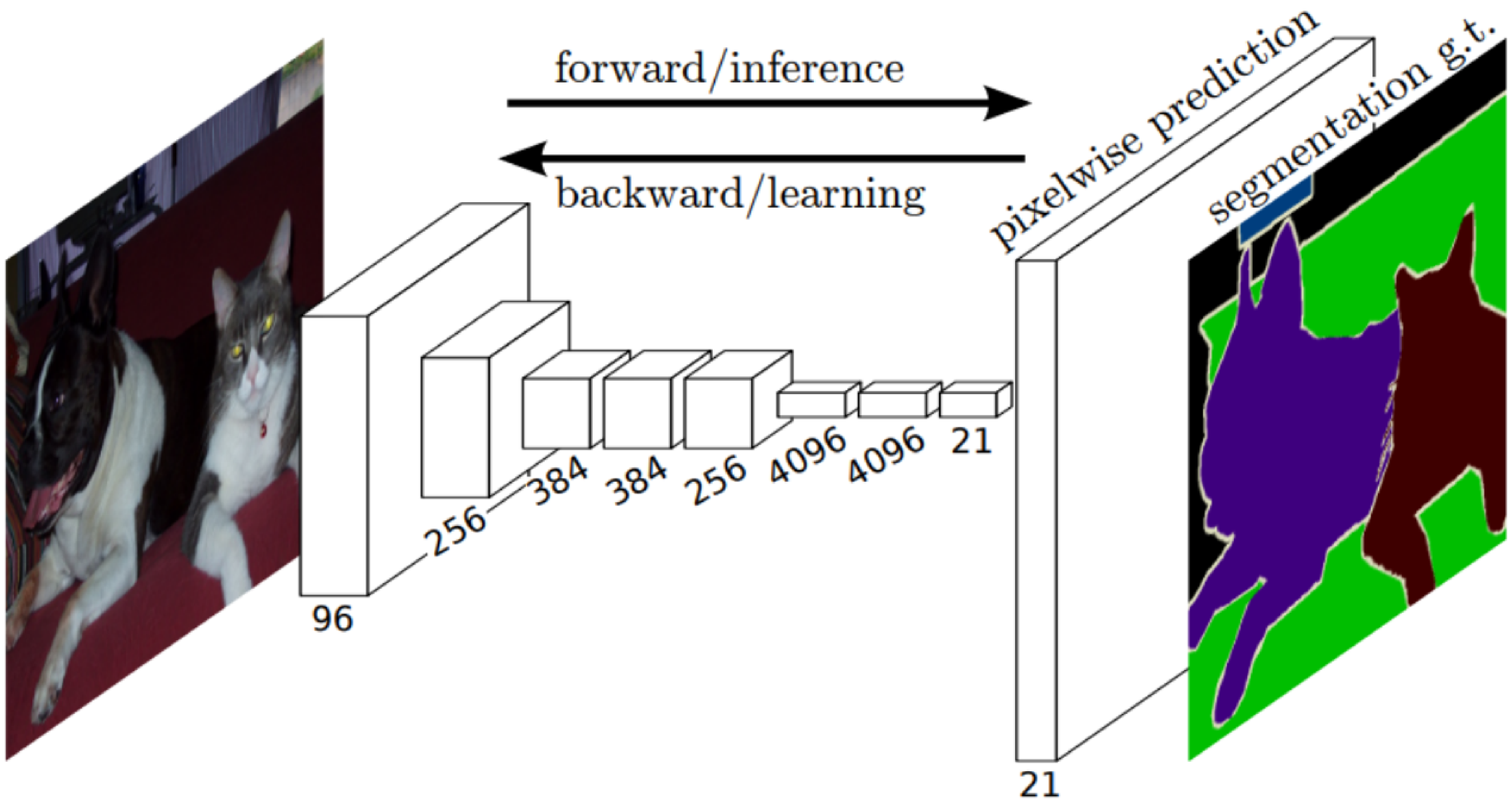


Figure 3: Fully convolutional networks can efficiently learn to make dense predictions for per-pixel tasks like semantic segmentation.

7.2 Fully convolutional networks

- fully connected layers can also be viewed as convolutions with kernels that cover their entire input regions
- doing so casts them into fully convolutional networks that take input of any size and output classification maps
- this transformation is illustrated in Figure 4

7.2 Fully convolutional networks

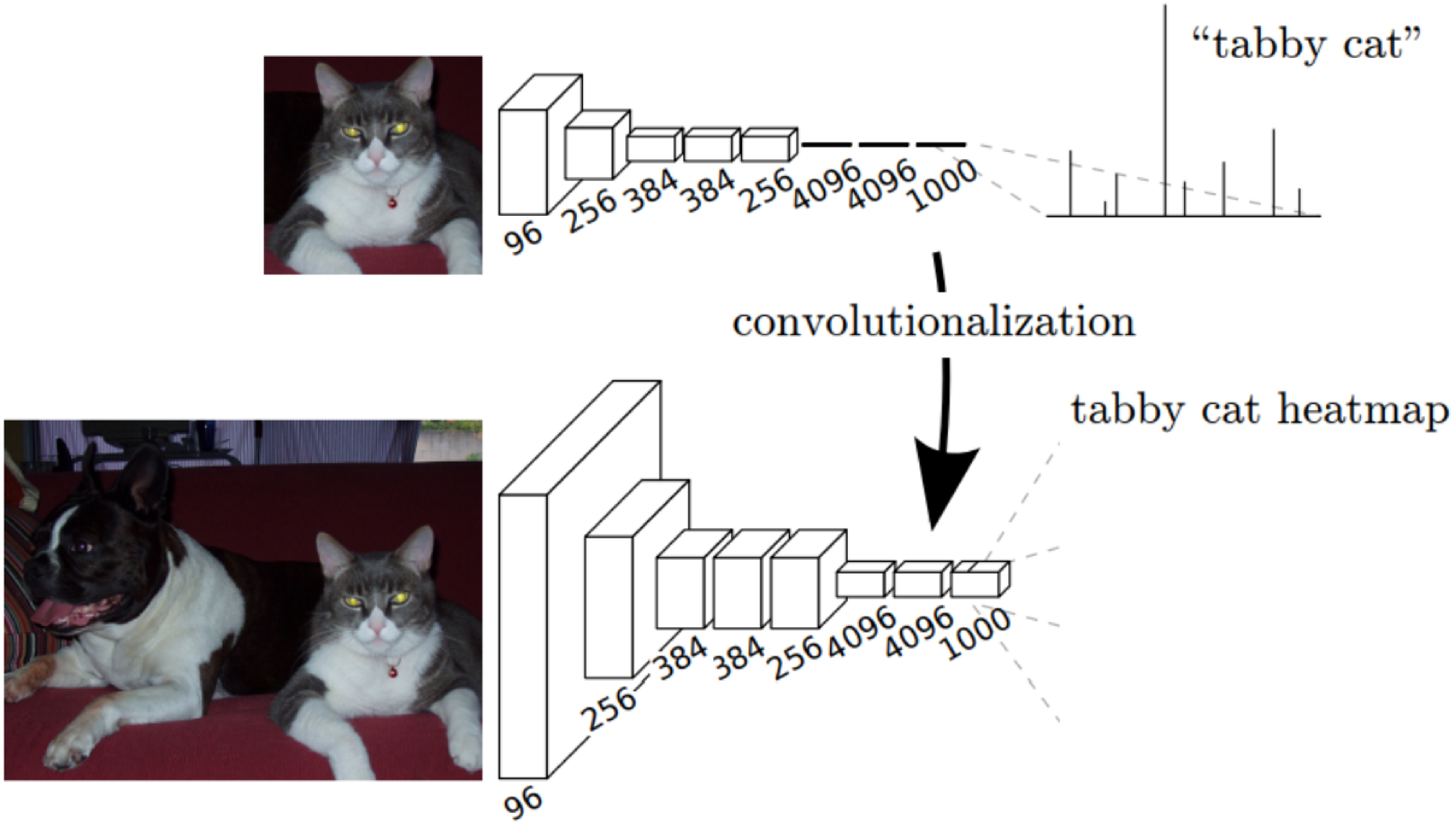


Figure 4: Transforming fully connected layers into convolution layers enables a classification network to output a heatmap.

7.2 Fully convolutional networks

- hierarchical training is used, where the network is initially trained only based on high-level features and then fine-tuned based on middle and low-level features
- this is important because it allows the network to also learn the middle and low-level details of the image, in addition to high-level ones
- initially, predict the segmentation mask from high-level features (FCN-32s)
- then, predict the segmentation mask from middle-level features (FCN-16s)
- finally, predict the segmentation mask from low-level features (FCN-8s)
- the whole process is illustrated in Figure 5
- an example showing the effectiveness of this approach is shown in Figure 6

7.2 Fully convolutional networks

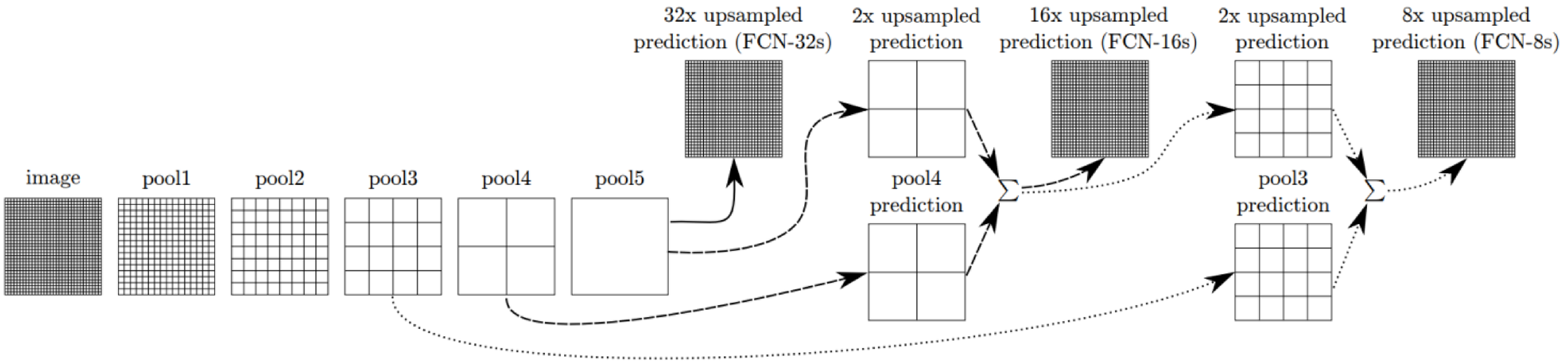


Figure 5: The FCNs learn to combine coarse, high-layer information with fine, low-layer information. Layers are shown as grids that reveal relative spatial coarseness. Only pooling and prediction layers are shown; intermediate convolution layers (including the converted fully connected layers) are omitted. Solid line (FCN-32s): the single-stream network, upsamples stride 32 predictions back to pixels in a single step. Dashed line (FCN-16s): combining predictions from both the final layer and the pool4 layer, at stride 16, lets the network predict finer details, while retaining high-level semantic information. Dotted line (FCN-8s): additional predictions from pool3, at stride 8, provide further precision.

7.2 Fully convolutional networks

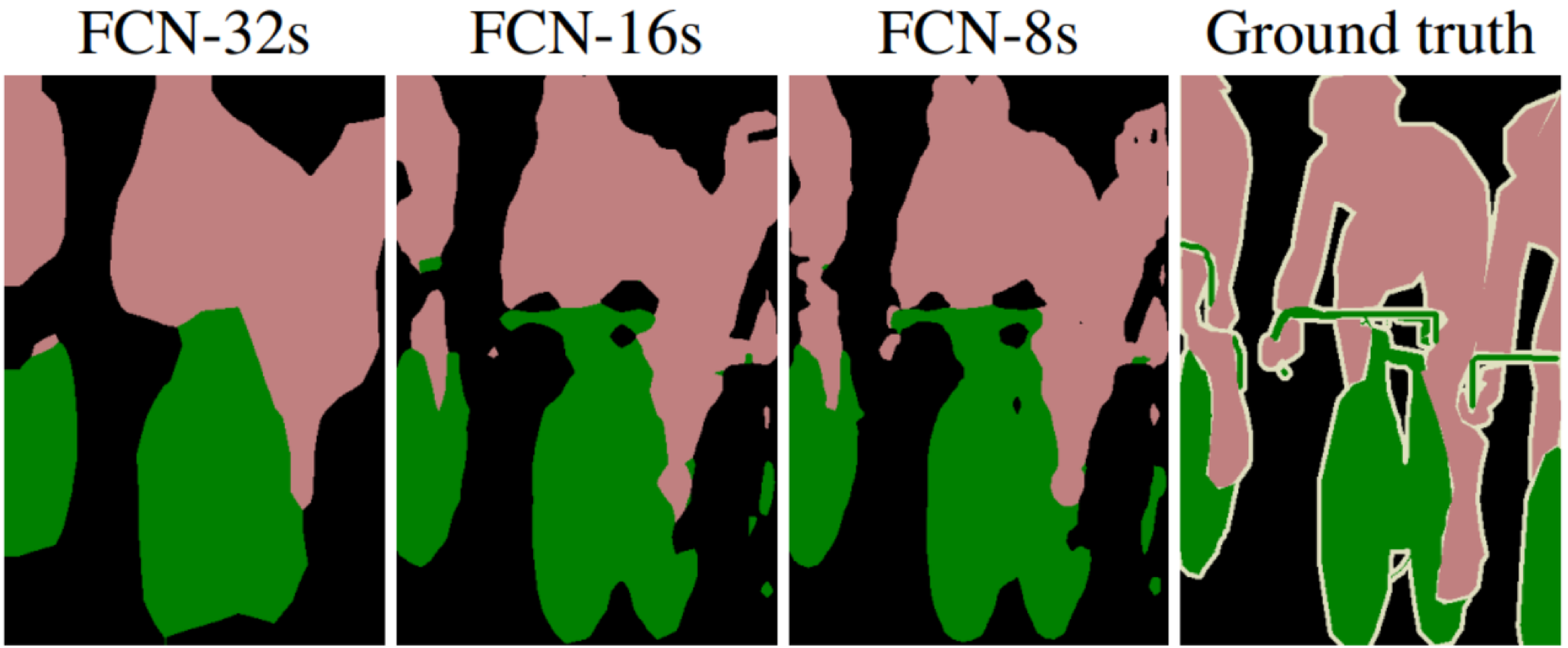


Figure 6: Refining FCNs by fusing information from layers with different strides improves segmentation detail. The first three images show the output from the 32, 16, and 8 pixel stride networks.

7.2 Fully convolutional networks

- now, we can discuss the training of the fully convolutional network
- the loss function and accuracy calculation are not essentially different from those in image classification
- because we use the output channel of the transposed convolutional layer to predict the class for each pixel, the channel dimension is specified in the loss calculation
- here, we use the softmax operation to predict the class, and we have 21 output channels because the Pascal VOC2012 dataset has 20 classes, and we add the background class
- the loss function is a per-pixel cross-entropy loss
- in addition, the accuracy is calculated based on the correctness of the predicted class, for all the pixels
- when predicting, we need to standardize the input image in each channel and transform the image into the four-dimensional input format required by the CNN
- to visualize the predicted class of each pixel, we map the predicted class back to its label color in the dataset